MINISTRY OF EDUCATION AND TRAINING
FPT UNIVERSITY

**CAPSTONE PROJECT PROPOSAL**

# A Web-Based System for Continuous Hotel Room Rate Monitoring and Short-Term Forecasting in Vietnam Using Machine Learning

**Dang Nguyen Minh An**
Student ID: 24MSE23217

**Supervisor: Dr. Le Thanh Hai**

A proposal submitted in conformity with the requirements
for the degree of Master of Software Engineering

Ho Chi Minh City, 2026

# Abstract

Room rates on Online Travel Agencies change as inventory, lead time and market conditions evolve, yet a single price snapshot cannot show whether a listed rate is rising or falling. This proposal describes a web-based system that repeatedly collects publicly visible Booking.com room rates for a selected cohort of 355 properties in five Vietnamese cities, forecasts short-term rate movements with machine learning, and serves the forecasts through two decision layers: a hotelier view centred on competitive-set monitoring and pricing-position alerts, and a consumer view providing informational booking-timing guidance. The system does not process bookings or guarantee a lowest price.

One modelling requirement shapes the entire data pipeline. Every price record carries two independent time coordinates, the observation timestamp and the stay date, so the dataset records how the rate for a fixed check-in date evolves as that date approaches. Lead time is derived from the two coordinates and becomes a central predictor. A durable MySQL work queue drives a Selenium crawler that separates sold-out nights from properties that cannot be booked at all and from dead listings. A reference definition calibrated per hotel and per check-in date keeps consecutive observations describing a comparable room and rate plan. From 18 August to 30 November 2026, the operator follows a fixed calendar and uses the scraper web interface to start one full-cohort run per day with nine near-term and three farther check-in dates, providing dense short-lead series while retaining longer-horizon coverage.

The required modelling deliverable is a reproducible regression pipeline for the 1, 3, 7 and 14-day horizons. Persistence and simple statistical baselines are compared with Random Forest and XGBoost; the tree models are tuned with time-aware cross-validation using RandomizedSearchCV followed by a focused GridSearchCV. Deep-learning and standalone classification models are optional extensions only if the core regression system is complete. Evaluation uses a strictly chronological split and adopts *Accuracy@20%* as the primary feasibility measure: at least 80 per cent of held-out predictions should have an absolute percentage error no greater than 20 per cent. MAE, RMSE, sMAPE, MAPE, $R^2$,

directional accuracy, SHAP attribution and two retrospective decision simulations provide complementary evidence.

# Contents

# List of Figures

# List of Tables

# List of Abbreviations

| | |
|---|---|
| **API** | Application Programming Interface |
| **DOM** | Document Object Model |
| **EDA** | Exploratory Data Analysis |
| **GSO** | General Statistics Office of Vietnam |
| **LSTM** | Long Short-Term Memory |
| **MAE** | Mean Absolute Error |
| **MAPE** | Mean Absolute Percentage Error |
| **ML** | Machine Learning |
| **MSE** | Master of Software Engineering |
| **OTA** | Online Travel Agency |
| **PMS** | Property Management System |
| **RMSE** | Root Mean Squared Error |
| **SHAP** | SHapley Additive exPlanations |
| **SQL** | Structured Query Language |
| **sMAPE** | Symmetric Mean Absolute Percentage Error |
| **UTC** | Coordinated Universal Time |
| **VNAT** | Vietnam National Authority of Tourism |
| **VND** | Vietnamese Dong |
| **XGBoost** | Extreme Gradient Boosting |

# CHAPTER 1. INTRODUCTION AND BACKGROUND

## 1.1 Problem Description

Vietnam had approximately 38,000 tourist accommodation establishments and 780,000 rooms at the end of 2023 (VNAT, 2024). The country's online-travel gross merchandise value increased from approximately USD 4 billion in 2023 to USD 5 billion in 2024 (Google et al., 2024), illustrating the growing importance of digital distribution. A property's published rate is visible to competitors and travellers, and the OTA display can change as inventory and partner rates appear or disappear. Hotels respond with dynamic pricing, adjusting rates according to remaining inventory, days to arrival and observed competitor behaviour: Abrate et al. (2012) documented systematic rate variation by lead time across European hotels, and Ivanov and Zhechev (2014) describe competitor rate monitoring as a standard input to pricing decisions.

Commercial rate-intelligence platforms such as RateGain and Lighthouse provide competitor-rate and market-insight capabilities (Lighthouse, 2025; RateGain, 2025), but adopting a specialised revenue-management platform also requires budget, configuration and operating expertise. Smaller operators may instead inspect competitor pages manually and price from a limited snapshot, which provides no longitudinal view of how the same stay date changes as arrival approaches. Travellers face the same information gap from the opposite side: the current listed rate alone does not reveal the recent trajectory for that property, room, rate plan and stay date.

### 1.1.1 Research Gap

Academic work covers determinants of hotel room prices, the effect of online reviews on room sales and aggregate tourism-demand forecasting (Ye et al., 2009; Zhang et al., 2011; Law et al., 2019). However, the literature reviewed for this proposal provides limited evidence on a combined system that repeatedly observes a comparable room and rate plan for a fixed stay date, forecasts its short-term trajectory, and evaluates the resulting signal in

the Vietnamese market. Four project gaps follow:

- **Longitudinal rate data with two time coordinates.** Published datasets record snapshots, whereas forecasting rate movement needs repeated observation of the same stay date over successive days.
- **Room comparability over time.** OTA listings reorder room options between sessions and add partner rates asynchronously, so a rule such as "take the cheapest room" silently switches products and corrupts the series.
- **A decision layer for independent operators.** Existing prototypes target the traveller or the revenue manager of a large chain, not the competitive-set view an independent hotel needs.
- **Vietnam-specific demand structure.** National holidays, lunar-calendar events and city-level festivals create local demand signals that a generic calendar does not encode.

## 1.2 Research Objectives

### 1.2.1 Main Objective

Design and build a web-based system for continuous Booking.com room-rate monitoring in five Vietnamese cities, forecast short-term listed rates at 1, 3, 7 and 14-day horizons using a selected regression model, and deliver the forecasts through a hotelier-facing competitive-set view and a consumer-facing informational booking-timing view. The model performs one task: predict the listed rate of hotel $h$ for check-in date $d$ as it will be observed $k$ days from now. The consumer view reads that prediction for one hotel and the hotelier view aggregates it over a competitive set, so two application layers rest on the same forecasting engine (Table 1.1).

### 1.2.2 Research Questions

1. How can a crawling pipeline collect Booking.com rates continuously over months while guaranteeing that consecutive observations of the same hotel and check-in date describe a comparable room and rate plan?

Table 1.1: SMART goal definition

| Criterion | Description |
|---|---|
| Specific | A durable crawling pipeline collecting Booking.com rates across multiple future check-in dates; a reproducible regression pipeline for four horizons; a dashboard exposing competitive-set monitoring and pricing-position alerts for hoteliers, and price history with informational booking-timing guidance for consumers |
| Measurable | Accuracy@20% $\geq$ 80% on the chronological test set; MAE, RMSE, sMAPE, MAPE, $R^2$ and directional accuracy reported by horizon, city and lead-time bucket; at least 100,000 price observations; technical crawl success rate $\geq$ 90% per run; parsed and saved counts reconciled for every successful item |
| Achievable | The durable crawler and reference-calibration workflow have passed a six-day pilot. The required modelling work uses scikit-learn, XGBoost, RandomizedSearchCV, GridSearchCV and SHAP on tabular data; deep learning is optional rather than a dependency |
| Relevant | Repeated public listing observations provide longitudinal evidence that a single price snapshot cannot supply, supporting competitor-rate analysis and transparent booking-timing signals for the Vietnamese context |
| Time-bound | A two-week warm-up from 16 to 31 August 2026 followed by the official three-month project window from 1 September to 30 November 2026; collection continues while feature, modelling and application work overlap |

2. How much do tuned Random Forest and XGBoost regressors improve on persistence and simple statistical baselines, and how does performance vary with lead time, city and horizon?
3. Which feature groups drive the forecast, measured by SHAP attribution and by ablation over the five groups?
4. To what extent do the forecasts support decisions, measured for hoteliers as detected pricing-position deviations during predicted high-demand periods, and for consumers as the listed-price difference between following the timing signal and acting immediately?

## 1.3 Scope of the Project

### 1.3.1 In Scope

Collection covers Ho Chi Minh City, Hanoi, Vung Tau, Da Lat and Phu Quoc, from Booking.com only, across a selected master list of 355 properties: 129 in Hanoi, 63 in Phu Quoc, 60 in Da Lat, 52 in Vung Tau and 51 in Ho Chi Minh City. The cohort manifest

records the source workbook and audit results; because the workbook does not contain an independently verified official star field, the proposal does not claim that every property belongs to a 3 to 5-star segment. Market tiers and competitive sets instead use Booking.com review score and review count. Every observation is collected under a fixed query context of 2 adults, 0 children, 1 room, 1 night, with currency forced to VND. Forecasts cover four horizons for a fixed check-in date. External enrichment uses a Vietnamese holiday and event calendar built for this project, weather forecasts from the Open-Meteo API (Open-Meteo, 2024), and monthly city-level tourist arrival statistics (VNAT, 2024; General Statistics Office, 2024).

### 1.3.2 Out of Scope

Excluded are properties outside the locked and audited cohort, homestays or short-term rentals identified as outside the target population during preflight, integration with a Property Management System or channel manager, any booking transaction, native mobile applications, and flight pricing. Evaluation is quantitative on held-out chronological data with no usability testing, and the system is built for academic research rather than commercial deployment.

### 1.3.3 Deferred to Future Work

Agoda collection and every feature depending on two OTAs observed at once were removed from the main scope: rate parity checking, cross-OTA price difference features, and the property-matching table. Maintaining one reliable crawler protects continuity better than splitting the three-month operating window across two changing websites. Standalone classification and sequence models such as LSTM or Transformer are also deferred unless regression, evaluation, the application and the written thesis reach their completion gates early. Adding these extensions later invalidates neither the dataset nor the forecasting architecture.

# CHAPTER 2. PROPOSED SOLUTION

## 2.1 Solution Description

The system has four parts: a durable collection pipeline, a room-comparability mechanism, a feature and labelling pipeline, and a forecasting engine feeding two application views. The data model described first constrains all of them.

### 2.1.1 The Two Time Coordinates

Every price observation carries two dates that vary independently: `observed_at`, the moment the crawler read the page, stored in UTC, and `checkin_date`, the night of stay the rate applies to. The rate for 20 December seen today differs from the rate for 20 December seen next week, so collapsing these into one date, which a single-snapshot crawl does implicitly, destroys the only signal a booking-timing or repricing decision can use. From the pair the pipeline derives `lead_time` = `checkin_date` − DATE(`observed_at`), computed at insert time and stored on the row. Each crawl run visits several future check-in dates per hotel, so after $N$ crawl days across $M$ check-in dates the dataset holds $N \times M$ points per hotel arranged as the grid in Figure 2.1.

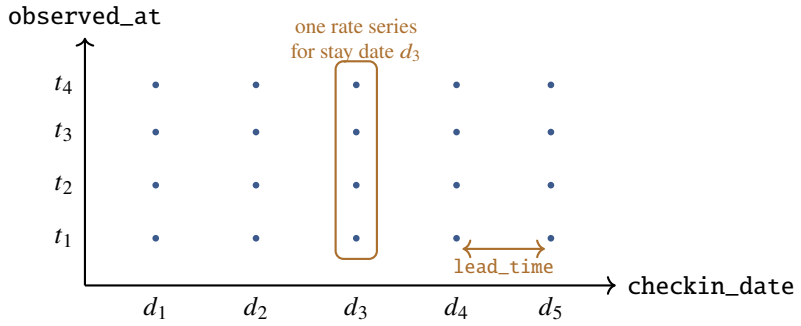


Figure 2.1: Observation grid. Each dot is one crawl of one hotel for one stay date; a column is a single rate series observed at successive times.

### 2.1.2 Data Collection

Booking.com renders room tables in JavaScript and inserts partner rates asynchronously, so collection uses Selenium with a real Chrome instance and waits for the number of rate

rows to stabilise before reading, which prevents recording a partial table as a complete one. Execution is split between two processes sharing MySQL as a durable queue: the API validates an uploaded workbook, creates one item per (hotel, check-in date) pair and returns immediately, while a separate worker claims items under a lease and writes a heartbeat while working. Closing the browser or restarting the API therefore does not affect a running job, and a worker crash does not leave items permanently claimed.

Each item records the counts that make completeness auditable, namely rows found, options parsed, options rejected and options written, and is marked `partial` when the parsed and saved counts disagree. Three outcomes stay distinct from each other and from a technical failure, because writing any of them as a zero price would inject a different fiction into the series: a night with no inventory becomes an observation with a null price and a sold-out status, a property-wide banner stating the hotel cannot be booked updates the property record and circuit-breaks its remaining queued dates without storing an observation, and a dead link or redirect to search results is classified as a collection error.

A six-day pilot on 10 properties validated parsing, persistence, reference calibration and outcome classification. Before the labelled-data clock starts, the final 355-property workbook is audited and load is benchmarked in stages, ending with 355 properties and five check-in dates. Pilot and benchmark observations remain available for pipeline audit but are excluded from model development unless they match the locked protocol and provenance requirements. The measured 355-by-five run completed in 6 hours 53 minutes 44 seconds, supporting a daily 12-date load within the 24-hour window. The locked calendar therefore covers all 355 properties once per day for nine near-term and three farther future check-in dates. Expired near dates are advanced by a fixed 28-day rule, while the three farther slots use fixed rotations to preserve longer lead-time coverage. Each day the operator opens the scraper web interface, uploads the locked hotel workbook, selects the 12 dates recorded for that day and starts the run manually. The calendar controls the sampling dates, while the operator controls every run.

### 2.1.3 Room Comparability

A price series is comparable over time only if consecutive observations describe the same product, yet Booking.com reorders rate options between sessions, so neither DOM position nor "cheapest available" identifies a stable product and building a reference room by hand for hundreds of properties is not workable. The system derives two hashes from attributes independent of price and DOM order: a room identity key over the canonicalised room name, occupancy, bed configuration and area, and a rate plan key over breakfast, free cancellation and the cancellation policy. Candidates accumulate per **(hotel, check-in date)** rather than per hotel, because each stay date is a separate series with its own inventory. A candidate becomes an approved reference only when it appears in at least three completed runs, reaches 80 per cent item coverage within that series, and matches exactly one option per item; until then it is excluded from training. Reference availability and crawl completeness are tracked separately, so a fully parsed item counts as a success even when its reference is unavailable for that night.

### 2.1.4 Features and Labels

Features fall into five groups: calendar features from the stay date, including public holidays, festivals and the Lunar New Year period; lead time in days and in buckets; price history for the same stay date, covering lags, rolling statistics, velocity and trailing extremes; competitive-set aggregates at the same observation time, including the ratio of own price to set mean and the share of the set sold out; and external context from weather forecasts and monthly city arrivals. Lag and rolling features hold `checkin_date` fixed and shift `observed_at`, since a lag across different stay dates would mix unrelated series, and weather must come from a forecast because realised weather is unavailable when the prediction is made.

Labels come from the series itself and need no manual annotation. For an observation at $(h, d, t)$ and horizon $k \in \{1, 3, 7, 14\}$ the pipeline locates the observation with the same hotel, the same check-in date and `observed_at` $= t + k$, then computes the future listed price and its change against the current price. A diagnostic direction label is *up* above +2%,

*down* below $-2\%$ and *stable* in between, with the band to be revisited after exploratory analysis; it evaluates the direction implied by regression and does not require a separate classifier. A row with no observation at $t + k$ drops out of that horizon's training set while remaining usable for others, and sold-out rows carry no price label.

### 2.1.5 Forecasting Engine and Two Views

The required experiment begins with transparent baselines and then compares two established tabular regression families (Table 2.1). Candidate selection uses chronological validation only. RandomizedSearchCV first explores a broad hyperparameter space; a focused GridSearchCV then refines the strongest region using time-aware folds. The held-out test period remains untouched until the pipeline and model choice are frozen.

Table 2.1: Required regression baselines and candidate models

| **Model** | **Family** | **Rationale** |
|---|---|---|
| Persistence and median | Baseline | Quantify whether a learned model improves on carrying the current rate forward and on a simple historical central tendency |
| Linear regression | Baseline | Provides an interpretable parametric reference and exposes whether nonlinear interactions are necessary |
| Random Forest | Bagging | Captures nonlinear interactions and reduces variance through an ensemble of trees (Breiman, 2001) |
| XGBoost | Gradient boosting | Strong performance on heterogeneous tabular data and sequential correction of residual errors (Chen and Guestrin, 2016; Grinsztajn et al., 2022) |

If the regression pipeline, evaluation, application and written thesis reach their completion gates early, LSTM or Transformer sequence models may be explored as a stretch comparison (Hochreiter and Schmidhuber, 1997; Vaswani et al., 2017; Zeng et al., 2023). A standalone up/down/stable classifier is a later optional extension. Neither is required to demonstrate the feasibility of the proposed solution.

In the hotelier view a property selects a competitive set of 5 to 15 comparable hotels defined by city, proximity in review score and a minimum review count; the dashboard shows current competitor rates, the forecast rate level of the set over the next 14 days, and an alert when the property's own rate deviates materially from the set or when the

model predicts a market movement around a holiday. In the consumer view the dashboard shows the observed rate history and forecast for one hotel and stay date and converts it into informational book-now or wait guidance. This signal reflects observed Booking.com listings only, performs no transaction and does not guarantee the lowest obtainable rate.

## 2.2 Software Architecture

The API and the crawl worker are separate processes coordinating only through the MySQL queue, so neither depends on the other staying alive (Figure 2.2). Two Next.js applications share one backend, an internal operations frontend and a product dashboard, and a single FastAPI service exposes `/api/scraper/*` for operations and `/api/dashboard/*` for the product views.

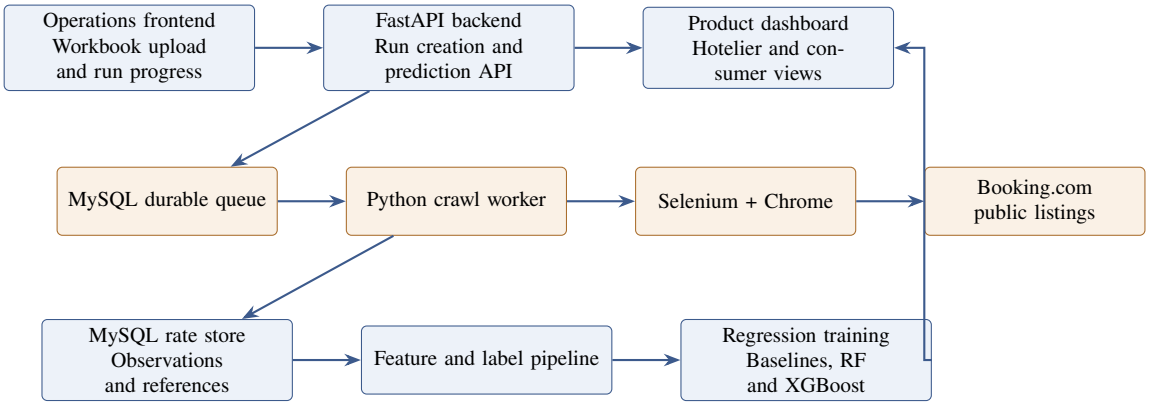


Figure 2.2: Target system architecture. Crawl runs are operator initiated; the durable worker continues independently after a run is created.

The schema separates near-static property attributes from the observation stream. A `hotels` table holds one row per property keyed by the Booking.com URL slug, which is stable across crawls and needs no property-mapping table while one OTA is in scope. A `price_observations` table holds one row per rate option per crawl of one hotel and stay date, carrying the two time coordinates, the precomputed lead time, price and tax fields, room and rate-plan attributes, both identity hashes and the availability status, indexed on `(hotel_id, checkin_date, observed_at)` with a unique constraint on `(crawl_run_item_id, room_option_index)`. Supporting tables cover the queue, room references and enrichment data, including a holiday and event calendar built for this

project that separates national from city-scoped entries so it joins directly to the city field on `hotels`.

## 2.3 Technology and Tools Used

Table 2.2: Technology stack

| **Layer** | **Technology** | **Rationale** |
|---|---|---|
| Frontend | Next.js | React-based, with routing and server rendering built in |
| Backend | FastAPI | Async handling, Pydantic validation, generated OpenAPI docs |
| Database | MySQL 8 | Relational fit for the hotel-to-observation hierarchy; the queue needs row-level locking and transactional commit |
| Job execution | Python worker over MySQL | Fewer moving parts than a broker for a single-worker workload; job state survives restarts |
| Collection | Selenium 4, BeautifulSoup | A rendering browser is required for JavaScript rates; CDP allows Vietnam geolocation and timezone emulation |
| ML | scikit-learn, XGBoost; TensorFlow 2 optional | Tabular regression is required; sequence modelling is a stretch goal |
| Tuning, analysis | Randomized search, grid search, SHAP | Time-aware hyperparameter search followed by additive feature attribution (Lundberg and Lee, 2017) |
| Operation and deployment | Local web application and scripts; VPS and Docker Compose planned | The operator uploads the locked workbook, selects the calendar dates and starts each run through the scraper interface; deployment packaging is added only after integration gates pass |

Restricting collection to Booking.com protects continuity of the time series, as Section 1.3.3 explains. MySQL is retained because the implemented schema and durable queue already use its transactional row locking. A database-backed worker replaces Celery and Redis so that an item's state and its observations commit or roll back together, and Scrapy is unnecessary because the target page requires JavaScript rendering. These implementation choices reduce the number of operating services without changing the forecasting methodology or evaluation design.

# CHAPTER 3. IMPLEMENTATION PLAN

## 3.1 Main Development Stages

The official project window is three months, from 1 September to 30 November 2026, preceded by a two-week full-cohort warm-up from 18 to 31 August. Data collection is the binding constraint because lag and rolling features require repeated observations over calendar time. Pilot runs and staged load tests are completed before the warm-up; eligible collection begins when the locked 355-property protocol starts on 18 August. Collection then continues in parallel with feature engineering, modelling, evaluation and application development. The phases overlap by design, and stretch work is removed before any core quality gate is weakened.

## 3.2 Expected Schedule

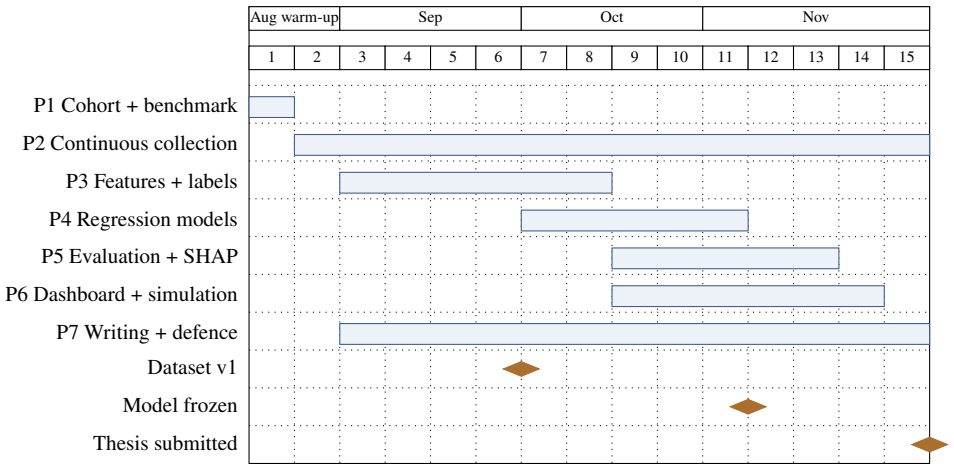


Figure 3.1: Project timeline: two-week August warm-up and the official September–November 2026 window. Collection overlaps every later phase.

## 3.3 Feasibility Assessment

Table 3.2 lists the risks that could stop the project and how each is handled.

Table 3.1: Development phases

| Ph. | Period | Activities | Deliverable |
|---|---|---|---|
| 0 | Before 16 Aug | Assemble the hotel master list; design the MySQL schema; implement the Booking.com collector, durable queue, reference calibration and holiday calendar; complete the six-day pilot | Validated local collection system and pilot audit |
| 1 | 16–17 Aug | Preflight the final workbook, create a versioned cohort manifest, audit property type and dead links, and complete staged benchmarks ending with 355 properties and five dates | Locked cohort candidate and capacity report |
| 2 | 18 Aug–30 Nov | Use the scraper web interface each day to run the locked calendar of nine near-term and three farther check-in dates across all 355 properties; track success, duration, backlog, completeness, continuity, reference readiness, missingness and selector health weekly | Longitudinal locked-cohort dataset and weekly quality reports |
| 3 | 1 Sep–10 Oct | Analyse distributions, sold-out rates and anomalies; integrate holidays, forecast weather and tourism statistics; define competitive sets; implement reproducible features and labels; prepare a chronological split with a 14-day embargo | Tabular dataset, data dictionary and label audit |
| 4 | 1 Oct–5 Nov | Establish persistence, median and linear baselines; train Random Forest and XGBoost; use RandomizedSearchCV followed by focused GridSearchCV with time-aware folds; freeze seeds, features, search results and selected model before test | Reproducible tuned regression pipeline |
| 5 | 20 Oct–15 Nov | Select on chronological validation; evaluate the frozen pipeline once on the final test period; compute Accuracy@20%, error metrics and diagnostic direction; run SHAP, ablation and decision simulations | Evaluation, attribution and simulation report |
| 6 | 20 Oct–22 Nov | Implement prediction serving and the two dashboard views; integrate the selected model; test the complete flow and clearly label consumer guidance as informational | Prediction API and product dashboard |
| 7 | Throughout; final 23–30 Nov | Maintain the proposal and thesis as decisions change; write methods, data, experiments, results and limitations; package reproducible artifacts and rehearse the defence | Final thesis, artifacts and defence materials |

Table 3.2: Risk assessment

| **Risk** | **Severity** | **Mitigation** |
|---|---|---|
| Booking.com changes its DOM and breaks selectors | High | Selector chains with fallbacks and a versioned selector set; per-item parsed and saved counts make a silent extraction failure visible as a count mismatch instead of a quietly empty result; selector checks repeated weekly |
| Gap in the time series from crawler downtime | High | Job state lives in MySQL, so an API or frontend restart does not lose a run; stale leases are reclaimed automatically; backups and a weekly continuity report expose gaps before model features are generated |
| IP rate limiting or blocking | Medium | Randomised delays between requests, one worker per address, no parallel runs; CAPTCHA and block events recorded as their own error codes so the rate is measurable |
| Insufficient data volume or throughput by the training deadline | High | Benchmark progressively before locking the protocol; use the measured 355-by-five duration to verify that a manual 355-by-twelve daily run fits within one day; monitor completion time and backlog daily, never overlap workers, and start feature work on accumulated data rather than waiting for collection to end |
| Room comparability drift over months | Medium | Reference definitions scoped per (hotel, check-in date), requiring three runs and 80% item coverage before approval; unapproved candidates excluded from training; ambiguous matches left unassigned |
| Model overfitting or leakage | High | Chronological split with a 14-day embargo; time-aware cross-validation; scalers and encoders fitted on training folds only; lags computed within a fixed check-in date; forecast weather only; final test used once after model freeze |

Current development and collection run on a personal machine with 16 GB RAM; tabular regression does not require a GPU. A virtual private server and Docker Compose remain deployment options after the local benchmark and integration gates pass, not assumptions required for data collection. The libraries in Table 2.2 are open source and enrichment comes from free public sources, so no licensed dataset or paid proxy service is required. The crawler accesses no user account or personal data, uses randomised delays, limits execution to one sequential worker, and stores property attributes and publicly listed room rates for academic research without commercial redistribution. Every official run is started by the operator through the web interface using the dates in the versioned calendar.

# CHAPTER 4. EXPECTED RESULTS

## 4.1 Deliverables

1. **A longitudinal Vietnamese hotel rate dataset** of at least 100,000 price observations across five cities, each row carrying both an observation timestamp and a stay date, with room and rate-plan identity hashes, reference match status, availability status and provenance fields, shipped with a data dictionary and a quality report.
2. **A reproducible regression pipeline** covering the 1, 3, 7 and 14-day horizons, with persistence and simple statistical baselines, tuned Random Forest and XGBoost candidates, stored configurations and held-out test metrics. At least one selected regression model must be served by the application.
3. **A comparative analysis report** giving performance per baseline, candidate and horizon, a breakdown by city, review-score tier and lead-time bucket, SHAP attribution for the selected model, and feature-group ablation results.
4. **A working system**: the FastAPI backend serving both route groups, the crawl worker over the MySQL durable queue, the operations frontend and the product dashboard. Deployment packaging is included if the local integration and reproducibility gates are complete.
5. **Two decision simulations** run retrospectively on the held-out period, one for the hotelier view and one for the consumer view.
6. **The thesis and defence materials.**

## 4.2 Performance Targets

Table 4.1 separates one feasibility target from reporting obligations. Accuracy@20% operationalises the supervisor's guidance that the proposed solution should aim for approximately 80 per cent accuracy or higher; it is not a reason to stop optimisation at the threshold. The remaining measures are reported without arbitrary pass values so that performance differ-

ences by horizon, lead-time bucket and city remain visible.

Table 4.1: Performance targets

| Metric | Target | Basis |
|---|---|---|
| Accuracy@20%, all horizons | ≥ 80% | Primary feasibility target: the share of test predictions whose absolute percentage error is no greater than 20% |
| MAE and RMSE | Report and minimise | Give errors in VND; RMSE makes large misses more visible |
| sMAPE, MAPE and $R^2$ | Report by segment | Provide relative error and explained variance without treating one aggregate as sufficient |
| Directional accuracy | Beat naive baselines | Evaluates the up/down/stable signal derived from regression; classifier metrics apply only if the optional classifier is built |
| Decision simulations | Report benefit and regret | Quantify listed-price opportunity, mean/median difference, downside and cases where the signal is worse than acting immediately; no guaranteed saving target |
| Crawl technical success | ≥ 90% per run | Counts technical failures only; sold-out nights and unbookable properties are valid outcomes |
| Parser persistence completeness | 100% for successful items | Parsed and saved option counts must reconcile; otherwise the item is partial rather than silently successful |
| Dataset size | At least 100,000 price observations | Volume target for the locked cohort; label coverage and reference readiness are reported separately |
| Service quality | Report p95 latency, freshness and retraining time | Measures operational feasibility without inventing unsupported pass thresholds before load testing |

## 4.3 Contributions

The project contributes a purpose-built longitudinal Vietnamese hotel-rate dataset with two independent time coordinates; a room-comparability mechanism designed to keep a series on a comparable room and rate plan without manual reference creation for every property; an evaluation of tuned tabular regression against transparent baselines for short-horizon forecasting; and a two-view rate-intelligence design evaluated quantitatively on held-out chronological data. The contribution is framed as a project-specific implementation and empirical study rather than a claim that no related dataset or forecasting system exists anywhere.

# CHAPTER 5. EVALUATION PLAN

## 5.1 Forecast Accuracy

Evaluation is entirely quantitative. The locked-cohort period is ordered by observation time and divided approximately 70/15/15 into training, validation and final test periods, with a 14-day embargo around boundaries to prevent the longest-horizon label from crossing a split. Time-aware cross-validation is used inside training for hyperparameter search, chronological validation selects the model, and the final test period is evaluated once after the feature pipeline and model choice are frozen. The primary feasibility measure is Accuracy@20%:

$$\text{Accuracy@20\%} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\left(\frac{|y_i - \hat{y}_i|}{y_i} \leq 0.20\right) \times 100\%.$$

The feasibility target is Accuracy@20% $\geq$ 80% at each reported horizon, while optimisation seeks the best validation performance rather than merely crossing the threshold. MAE gives the typical error in VND, RMSE exposes large misses, sMAPE and MAPE give relative error, and $R^2$ reports explained variance. Results are broken down by horizon, lead-time bucket, city and review-score tier because an acceptable aggregate can hide an unusable last-minute or city-specific result.

The direction implied by each regression prediction is also evaluated over up, down and stable, reporting directional accuracy and a per-horizon confusion matrix. Persistence, historical median and a simple linear model provide forecast baselines; a same-weekday value may be added where coverage permits. Precision, recall and F1 become required only if the optional standalone classifier is implemented.

## 5.2 Interpretability

SHAP analysis (Lundberg and Lee, 2017) identifies the ten most influential features, the shape of each effect, and whether the drivers differ by city and review-score tier. An ablation

study retrains the model with one feature group removed at a time, measuring what a group is worth overall rather than its attribution within a fitted model. The competitive-set group is what this project adds beyond price-history forecasting, so its ablated contribution is a named result.

## 5.3 Decision Value

Two retrospective simulations run over the test period. The **pricing opportunity analysis** takes the hotelier view: it computes the forecast seven-day movement of each competitive set, flags properties whose own rate moved against that movement or stayed flat while the set was predicted to rise, and quantifies the gap in VND against the realised set level and how many days ahead an alert would have fired. It cannot claim added revenue, since the dataset holds rates and not bookings.

The **booking-timing simulation** takes the consumer view: it treats the first observation in each series as the moment a traveller starts watching, converts each later forecast into a hypothetical book-now or wait decision, and compares the listed rate selected by that policy with acting immediately and with the retrospective minimum. Reported measures are mean and median listed-price difference, downside or regret, the share of cases where waiting was worse, and breakdowns by city and lead-time bucket. This is a signal-quality simulation, not evidence of an actual booking, realised saving or guaranteed lowest rate.

## 5.4 Data and System Quality

Pipeline measures are the crawl success rate per run, counting technical failures only; persistence completeness, the share of items where parsed and saved counts agree; series continuity, the largest gap per series, since gaps invalidate nearby lag features; reference readiness against the 80 per cent gate; and missingness over time. Service measures are p95 API latency, data freshness, retraining time and worker recovery under fault injection.

## 5.5 Limitations

All rate data comes from Booking.com, so findings describe listed-rate behaviour on that platform. Only observations from the locked 355-property cohort under the final protocol are eligible for modelling; the earlier 10-property pilot and load benchmarks serve pipeline audit only. Full-cohort warm-up observations from 18–31 August may enter the beginning of the training period only when their protocol and provenance match the official collection. The three-month official window covers only part of an annual cycle, so yearly seasonality cannot be established. The dataset holds rates rather than searches, bookings, occupancy or revenue; both simulations therefore measure signal quality rather than realised business or consumer outcomes. The cohort is purposively selected by city rather than probability-sampled from a national registry, and property-type conclusions are limited to the audited manifest.

# REFERENCES


Abrate, G., Fraquelli, G., and Viglia, G. (2012). Dynamic pricing strategies: Evidence from European hotels. *International Journal of Hospitality Management*, 31(1), 160–168.

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.

Chen, T., and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794).

General Statistics Office of Vietnam (2024). *Statistical data on tourism and accommodation services*. Retrieved August 10, 2026, from https://www.gso.gov.vn/

Google, Temasek, and Bain & Company (2024). *e-Conomy SEA 2024: Profits on the rise, harnessing SEA's advantage—Vietnam country report*. Retrieved August 16, 2026, from https://services.google.com/fh/files/misc/vietnam_e_conomy_sea_2024_report.pdf

Grinsztajn, L., Oyallon, E., and Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems 35, Datasets and Benchmarks Track*.

Hochreiter, S., and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.

Ivanov, S., and Zhechev, C. (2014). Hotel revenue management: A critical literature review. *Tourism: An International Interdisciplinary Journal*, 60(4), 355–376.

Law, R., Li, G., Fong, D. K. C., and Han, X. (2019). Tourism demand forecasting: A deep learning approach. *Annals of Tourism Research*, 75, 410–423.

Lighthouse (2025). *Hotel rate intelligence and market insight platform*. Retrieved August 10, 2026, from https://www.mylighthouse.com/

Lundberg, S. M., and Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30* (pp. 4765–4774).

Open-Meteo (2024). *Open-Meteo: Free weather API*. Retrieved August 10, 2026, from https://open-meteo.com/

RateGain Travel Technologies (2025). *Rate intelligence and revenue management solutions*. Retrieved August 10, 2026, from https://rategain.com/

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems 30* (pp. 5998–6008).

Vietnam National Authority of Tourism (2024). *Vietnam aims to attract 17–18 million international visitor arrivals in 2024*. Ministry of Culture, Sports and Tourism. Retrieved August 16, 2026, from https://vietnamtourism.gov.vn/en/post/19441

Ye, Q., Law, R., and Gu, B. (2009). The impact of online user reviews on hotel room sales. *International Journal of Hospitality Management*, 28(1), 180–182.

Zeng, A., Chen, M., Zhang, L., and Xu, Q. (2023). Are transformers effective for time series forecasting? In *Proceedings of the 37th AAAI Conference on Artificial Intelligence*

(pp. 11121–11128).

Zhang, Z., Ye, Q., and Law, R. (2011). Determinants of hotel room price: An exploration of travelers' hierarchy of accommodation needs. *International Journal of Contemporary Hospitality Management*, 23(7), 972–981.